

PatchPilot

A Tool-Using Agent for Python Debugging and Repair

Agentic AI Master's Project Report

A bounded repair system that reproduces failures, inspects code through restricted tools, applies controlled patches, and reports success only after executable test verification.

Team Members

Tivdzua Lubem Noah

Gisele Wiykiynyuy

Project type: Agentic software repair prototype
Primary language: Python
Model backend: Ollama with `qwen2.5-coder:1.5b`
Final artifact: Reproducible repository, live demo, evaluation results, and report

Final Report

July 9, 2026

Abstract

PatchPilot is a bounded Agentic AI system for Python debugging and repair. The system treats repair as an interaction between a language-model policy and a restricted software environment. Given a task with failing tests, PatchPilot prepares an isolated workspace, runs the tests, searches and reads relevant source files, applies a controlled patch, and verifies the result with pytest. A run succeeds only when the current workspace revision passes the configured test command.

The project deliberately avoids unrestricted repository editing. Task manifests define editable and forbidden paths, tool calls are schema-validated, tests are protected from modification, patch attempts are budgeted, failed edits can be restored, and every run produces a machine-readable trace. This design makes the agent useful as a research prototype for bounded repair rather than as an unguarded code-generation system.

Evaluation used four tracks. The controlled benchmark verified the end-to-end repair loop on 12 curated tasks and was saturated by all compared variants. The main ablation benchmark used 20 real mutmut-generated tasks. On that benchmark, the full agent repaired 8 of 20 tasks, compared with 6 of 20 for one-shot repair and 5 of 20 for a no-retry ablation. All three conditions had a measured invalid patch rate of 0%. A small QuixBugs smoke test repaired 3 of 8 public algorithmic tasks. SWE-bench Lite was attempted as a feasibility check with the official harness, but a full local run was not completed because of WSL/Docker filesystem instability. These results support a modest, honest conclusion: the full tool-using loop produced the strongest result on the primary benchmark, but the sample size is too small for a statistically significant superiority claim.

Project Motivation

Modern code models can produce plausible patches, but plausible text is not the same as a verified repair. A repair assistant needs to interact with the repository, reproduce the failure, inspect the right files, make a limited edit, and check that the current code actually passes tests. PatchPilot was built around that requirement.

The project explores a practical agentic pattern: the model is responsible for choosing the next repair action, while the runtime is responsible for enforcing boundaries. This separation matters. It prevents the language model from directly mutating arbitrary files and makes the repair process inspectable through tool observations and traces. The result is a small but complete debugging agent that demonstrates planning, action, observation, reflection, rollback, and verification in a reproducible Python setting.

Scope and Goals

PatchPilot targets small Python repositories with executable pytest suites. The goal is not to solve arbitrary real-world software engineering issues, nor to compete with large production repair systems. The goal is to build and evaluate a bounded agent that can perform a complete test-driven repair cycle under explicit safety constraints.

The final system was expected to satisfy five core requirements:

- reproduce a failing task in an isolated workspace;

- inspect source code through restricted repository tools;
- apply a minimal source-code repair without editing protected tests;
- verify success through a passing pytest run rather than model self-report;
- record enough trace information for the run to be audited after completion.

The implemented project meets those requirements and adds a live Streamlit demonstration, a mutmut-based benchmark generation pipeline, evaluation scripts, committed result summaries, and final report assets.

Related Work and Tools

PatchPilot sits between automatic program repair and current language-model agent work. QuixBugs is a widely used small-bug repair benchmark with Python and Java programs drawn from programming-challenge tasks [1]. SWE-bench evaluates whether language models can resolve real GitHub issues in Python repositories [2]; SWE-bench Lite provides a smaller 300-task subset intended to reduce evaluation cost while preserving the benchmark’s real-issue setting [3]. PatchPilot does not claim a completed SWE-bench result, but the harness feasibility check helped frame the gap between a course-scale prototype and a full software engineering benchmark.

The primary generated benchmark uses mutmut, a Python mutation-testing tool that creates code mutants and reports which mutants are killed by the test suite [4]. PatchPilot converts killed mutants into repair tasks: each generated task contains a repository state with a real seeded defect and a test suite that exposes that defect. This gives the project a more systematic benchmark than hand-authored examples alone.

The live repair policy uses Qwen2.5-Coder through Ollama. Qwen2.5-Coder is a code-focused model family described by Hui et al. [5]. The small 1.5B model was used because it was practical for local CPU-oriented execution. The report therefore evaluates the PatchPilot architecture under a constrained local model setting rather than under a large hosted frontier model.

System Design

PatchPilot is organized as a bounded loop. A benchmark task defines the repository, the test command, editable paths, and forbidden paths. The runner copies the task into a fresh workspace. The policy then chooses the next action from a small tool set, and the executor validates and runs that action. The observation is written back into agent state, and the loop continues until the run succeeds, escalates, or exhausts its budget.

Task manifests

Each task is represented by a manifest. The manifest describes the task identifier, repository location, test command, allowed editable paths, protected paths, and metadata such as category and difficulty. This format keeps benchmark execution separate from the original repository. It also lets the same repair loop run over hand-authored tasks and mutmut-generated tasks.

The most important safety fields are the path controls. Source files under `allowed_paths` may be read and patched. Test files and other forbidden locations are protected. This prevents the agent from solving a task by weakening or rewriting the tests.

PatchPilot Bounded Repair Loop

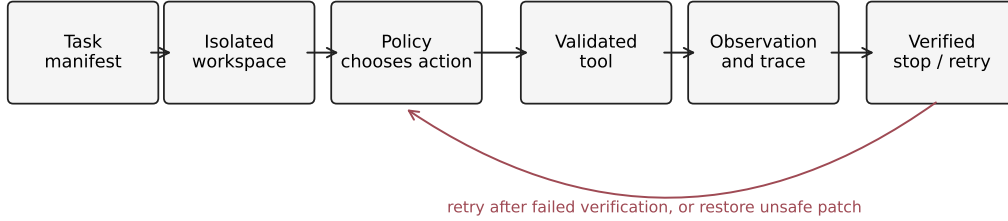


Figure 1: PatchPilot repair loop. The policy chooses actions, but the runtime validates and executes them inside an isolated workspace. Failed verification can trigger retry or restoration.

Tool interface

The live repair loop uses a compact set of tools: run tests, search code, read a file, apply a patch, restore a file, and finish. The policy must choose from these actions rather than emitting arbitrary shell commands. The executor validates arguments before any action is performed. This makes each step easier to inspect and reduces the attack surface of the prototype.

Repair policy

The language-model policy is prompted to act as a test-driven repair agent. It first observes the failing tests, then searches for relevant source code, reads the candidate file, and proposes a small source change. The implementation includes focused failure targeting so that the prompt can emphasize the function or source region implicated by the current failing test output. If a patch fails verification, the runtime can restore the changed file before the next attempt. This avoids stacking multiple bad edits on top of one another.

The policy is intentionally conservative. It is not allowed to return test edits as a repair. It is also constrained to source changes inside the task’s editable paths. A final response is accepted only after a successful test run.

Traceability

Every run records the sequence of tool calls, observations, patch attempts, changed files, and final status. These traces are useful for debugging the system and explaining why a run succeeded, escalated, or exhausted its budget. Raw trace files are runtime artifacts and are not committed to the repository, but summarized results are committed under `results/`.

Implementation

The implementation is a Python package under `src/patchpilot/`. The agent components cover policy execution, control-loop state, tool execution, and trace recording. Benchmark utilities load task manifests and prepare isolated workspaces. Evaluation utilities collect run-level metrics and produce summary files.

The repository also contains a Streamlit frontend for demonstration. The UI is not only a static dashboard; it can launch a local repair run when Ollama and the selected model are

available. It loads both the controlled benchmark and the mutmut-generated tasks. Users can inspect task metadata, source and tests, run PatchPilot, view the tool timeline, inspect the diff, and confirm final pytest verification.

The project was kept reproducible by committing source code, benchmark manifests, seed projects, generated task repositories, result CSVs, and generated report assets. Runtime artifacts such as workspaces, raw traces, caches, cloned external repositories, and temporary harness outputs are ignored.

Demonstration Interface

The demo page was included to make the repair workflow inspectable without requiring users to read raw trace files. A user selects a task, inspects the broken source and tests, starts a bounded repair run, and then reviews the evidence produced by the run. The hosted frontend is available online, while reliable live repair requires a local environment with Ollama access.

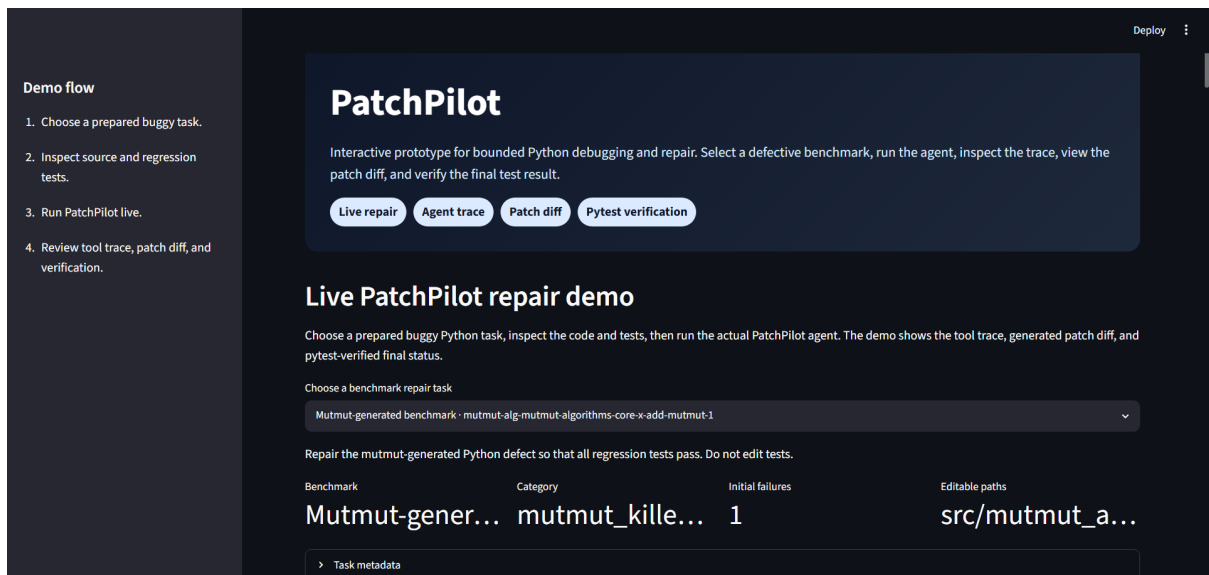


Figure 2: PatchPilot demonstration page. The interface presents task selection, source and test inspection, run controls, and verification-oriented evidence for the live repair workflow.

Table 1: Demonstration page responsibilities.

UI area	Purpose
Task selector	Selects a controlled or mutmut-generated repair task.
Source and tests	Shows the broken source, regression tests, task metadata, and editable paths.
Run control	Starts a bounded PatchPilot run through the configured local model backend.
Trace timeline	Presents the sequence of validated tool actions and observations.
Patch diff	Displays the generated source edit and changed files.
Verification panel	Reports the final pytest-verified outcome.

Benchmark Design

The evaluation uses four tracks, each serving a different purpose. The controlled benchmark is a sanity check for the full system. The mutmut-generated benchmark is the primary ablation benchmark. QuixBugs provides a small external smoke test. SWE-bench Lite is documented as a feasibility check only.

Controlled benchmark

The controlled benchmark contains 12 small repair tasks. These tasks verify that the runner can prepare workspaces, execute tests, apply patches, protect tests, and stop only after verification. All compared conditions reached 12 of 12 on this benchmark, so it is best interpreted as an end-to-end reliability check rather than a differentiating benchmark.

Mutmut-generated benchmark

The mutmut benchmark is the primary comparison. A seed project under `benchmark_seeds/mutmut_algorithms/` was mutated with mutmut. Killed mutants were converted into PatchPilot tasks under `generated_benchmarks/mutmut_algorithms/`. Each task contains a mutated source state and a test suite that exposes the defect. The final committed benchmark contains 20 tasks.

This benchmark is stronger than the controlled set because tasks are generated from mutation-testing output rather than manually tuned examples. It is still small, so statistical conclusions must be conservative.

QuixBugs smoke test

The QuixBugs smoke test uses eight public algorithmic tasks. It is not the primary benchmark, and it is not large enough to support broad claims about generalization. It is included to show that the repair loop can operate outside the internal task family.

SWE-bench Lite feasibility

SWE-bench Lite was attempted through the official harness because it is a recognized benchmark for real issue resolution. The dataset and harness were reachable, and a no-op prediction path was checked. A later local smoke attempt with a non-empty patch ran into WSL/Docker filesystem instability. The run was stopped to avoid damaging the local environment. For that reason, this report does not claim a completed SWE-bench Lite evaluation.

Table 2: Benchmark tracks used in the project.

Benchmark	Size	Role	Notes
Controlled benchmark	12	End-to-end reliability check	Hand-authored repair tasks with executable pytest suites.
Mutmut-generated benchmark	20	Primary ablation benchmark	Killed mutation-testing mutants converted into PatchPilot task manifests.
QuixBugs smoke	8	External generalization check	Small public algorithmic repair smoke test.
SWE-bench Lite harness smoke	1	Feasibility check	Official harness was reached, but full execution was blocked by local WSL/Docker I/O instability.

Experimental Conditions and Metrics

The final comparison uses three conditions:

- **Full agent:** the normal PatchPilot loop with test execution, source inspection, patching, verification, restoration, and retry behavior within the configured budget.
- **One-shot:** a baseline that attempts a direct patch without the full iterative repair loop.
- **No-retry:** an ablation with reduced retry behavior and tighter repair dynamics.

The main metric is repair rate, defined as the fraction of runs that end with the repository passing the configured test command. Full-suite pass rate is equivalent in the final summaries because a repair is only counted as successful after verification. Safety is measured through invalid patch rate. Additional diagnostic metrics include escalations, budget exhaustion, patch attempts, steps, tool calls, and elapsed time.

Results

The controlled benchmark confirms that the implementation works end to end. However, it does not separate the full agent from the baselines because all three conditions solve all 12 tasks. The mutmut benchmark is more informative. The full agent repairs 8 of 20 tasks, compared with 6 of 20 for one-shot and 5 of 20 for no-retry. The full agent therefore has the highest observed repair rate on the main benchmark, but the difference should be described as directional rather than conclusive.

The status breakdown explains part of the ablation behavior. One-shot repair often escalates because a single patch attempt is not enough when the model fails to infer the correct edit immediately. The no-retry condition is more likely to reach a budget limit. The full agent still fails on most mutmut tasks, but it recovers enough additional cases to reach the highest repair count.

The invalid patch rate is 0% in the final summaries. This should be interpreted carefully. It means the final evaluation did not count unsafe or structurally invalid patch outcomes under the implemented criteria. It does not mean the model always proposes correct code. In practice, the runtime rejects or restores failed edits and the agent can still escalate or exhaust its budget.

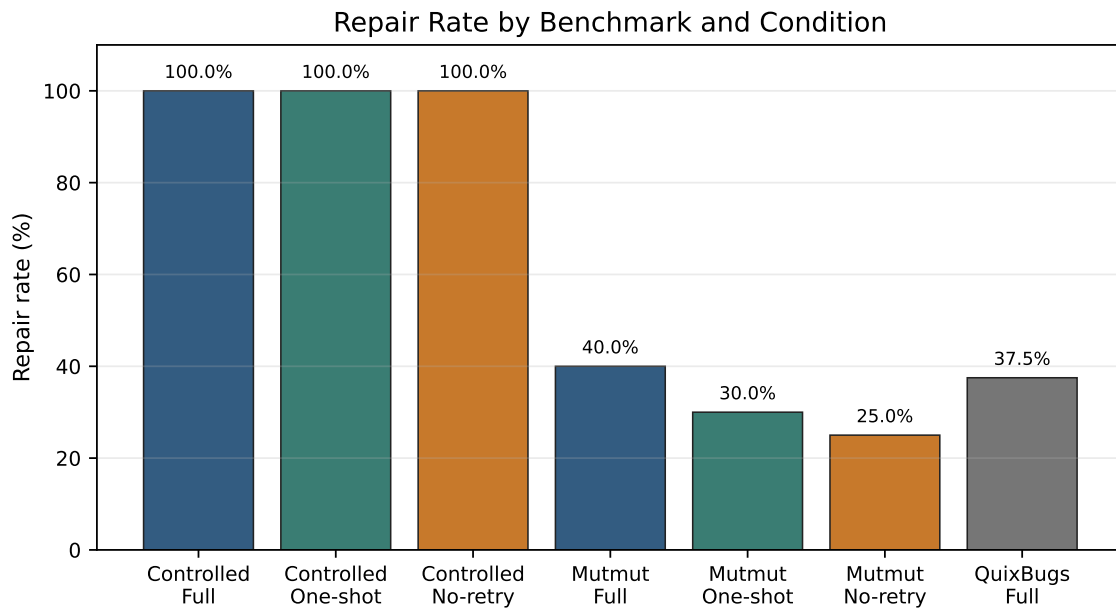


Figure 3: Repair rate across the final benchmark tracks and conditions. The controlled benchmark is saturated, while mutmut provides the main ablation comparison.

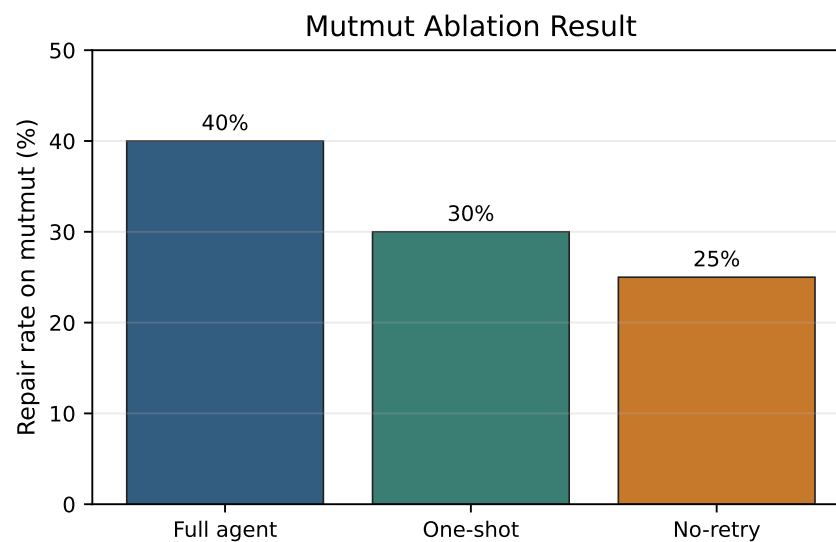


Figure 4: Mutmut ablation result. The full agent has the highest repair rate, but the 20-task sample limits statistical power.

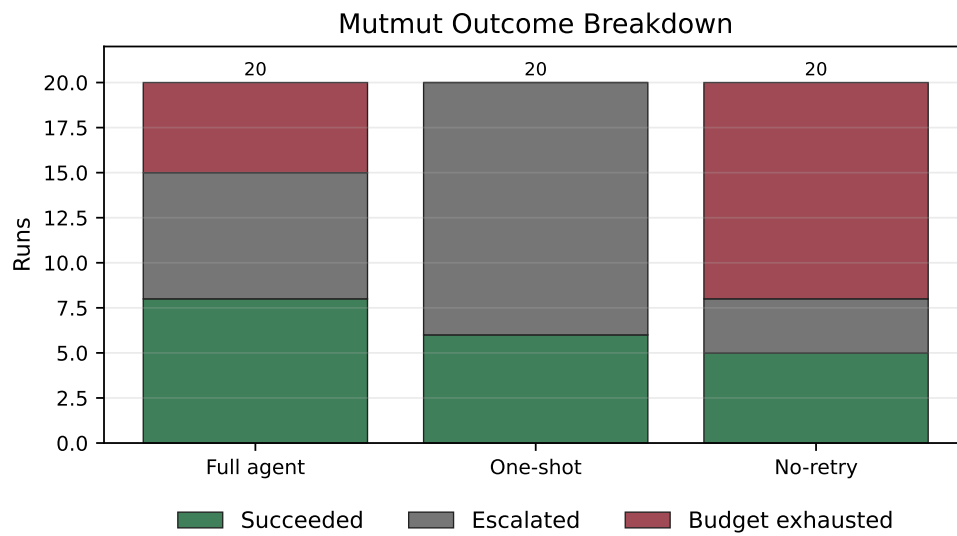


Figure 5: Outcome breakdown on the mutmut benchmark. The no-retry condition has more budget exhaustion, while the one-shot condition escalates more often.

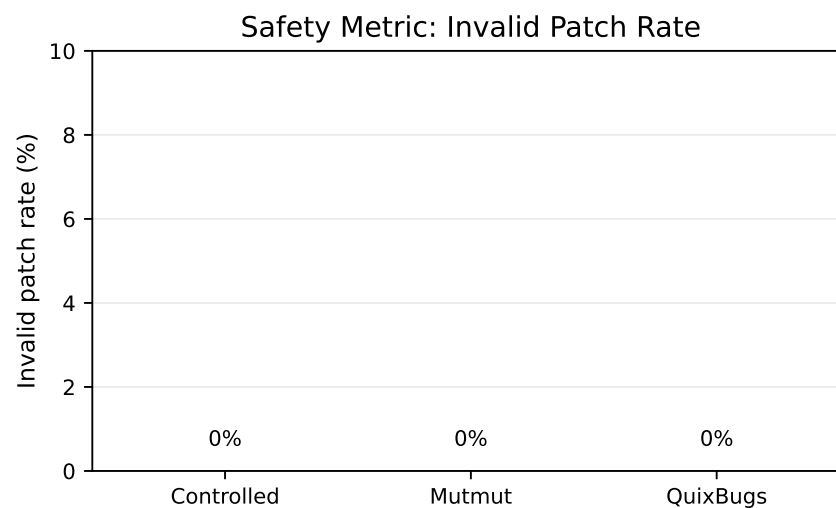


Figure 6: Invalid patch rate across evaluated tracks. In the final summaries, no condition produced a counted invalid patch. This is a safety result, not a claim that the system cannot generate incorrect patches.

Mutmut Task-Level Success Matrix

Task 01	pass	pass	pass
Task 02	pass	pass	pass
Task 03	pass	pass	pass
Task 04	pass	pass	pass
Task 05	pass	pass	pass
Task 06	pass	pass	fail
Task 07	pass	fail	fail
Task 08	pass	fail	fail
Task 09	fail	fail	fail
Task 10	fail	fail	fail
Task 11	fail	fail	fail
Task 12	fail	fail	fail
Task 13	fail	fail	fail
Task 14	fail	fail	fail
Task 15	fail	fail	fail
Task 16	fail	fail	fail
Task 17	fail	fail	fail
Task 18	fail	fail	fail
Task 19	fail	fail	fail
Task 20	fail	fail	fail
	Full	One-shot	No-retry

Condition

Figure 7: Task-level mutmut success matrix reconstructed from paired outcomes. In the final paired results, no baseline solves a task that the full agent misses.

Table 3: Final evaluation summary. Controlled tasks are used as a reliability check; mutmut is the primary ablation benchmark.

Benchmark	Condition	Runs	Successes	Rate	Invalid	Interpretation
Controlled	Full agent	12	12	100.0%	0.0%	Saturated end-to-end reliability check.
Controlled	One-shot	12	12	100.0%	0.0%	Saturated baseline on the controlled set.
Controlled	No-retry	12	12	100.0%	0.0%	Saturated reduced-loop baseline.
Mutmut	Full agent	20	8	40.0%	0.0%	Primary tool-use result.
Mutmut	One-shot	20	6	30.0%	0.0%	Baseline without the full repair loop.
Mutmut	No-retry	20	5	25.0%	0.0%	Reduced retry and reflection budget.
QuixBugs	Full agent	8	3	37.5%	0.0%	External smoke check.
SWE-bench Lite	Harness smoke	1	0	0.0%	n/a	Feasibility attempt only; no completed benchmark claim.

Table 4: Mutmut outcome breakdown by condition.

Condition	Succeeded	Escalated	Budget exhausted	Total
Full agent	8	7	5	20
One-shot	6	14	0	20
No-retry	5	3	12	20

Statistical Analysis

The mutmut comparison uses paired outcomes because the same 20 tasks are evaluated under each condition. The exact paired sign-test results are shown in Table 5. The full agent has two successes that one-shot does not have and three successes that no-retry does not have. No baseline has a unique success over the full agent in the final paired outcomes.

The p-values are not below 0.05. This is expected with only two or three discordant pairs. The honest conclusion is that the full agent performed best in the observed benchmark, but the experiment does not provide large-sample statistical proof. A stronger claim would require more tasks, more seeds, and repeated runs across additional models.

Table 5: Paired exact sign-test results on the 20 mutmut tasks.

Comparison	First-only	Second-only	Discordant	p-value
Full agent vs one-shot	2	0	2	0.5000
Full agent vs no-retry	3	0	3	0.2500
One-shot vs no-retry	1	0	1	1.0000

Discussion

PatchPilot shows that a small local model can be placed inside a structured repair environment and produce verified repairs on a subset of tasks. The most important contribution is the bounded workflow, not the raw repair rate. The system demonstrates that model-generated repair can be wrapped in path controls, tool validation, rollback, budgets, traces, and test-based acceptance.

The mutmut results suggest that iterative tool use is useful. The full agent repairs more tasks than one-shot and no-retry baselines. The extra successes are plausible because the full loop can inspect failures, restore bad edits, and continue after a failed verification attempt. At the same time, the repair rate remains modest. Most mutmut tasks are not solved by the small local model under the current prompting and budget. This leaves clear room for stronger localization, better patch synthesis, larger models, and broader benchmarks.

The QuixBugs smoke result is also modest. Repairing 3 of 8 tasks shows that the agent is not only hard-coded to the internal benchmark, but the sample is too small for a broad generalization claim. The SWE-bench attempt is included transparently because it is relevant to the project scope, but it is reported only as a feasibility exercise.

Reproducibility

The repository is organized so that another user can inspect the implementation, run the demo locally, regenerate report assets, and rerun the evaluation scripts. The root README contains installation, quality-gate, demo, benchmark-generation, and evaluation commands. The committed result summaries are under `results/`. Generated report figures and tables are under `docs/report_assets/`. Runtime outputs such as workspaces and traces are intentionally ignored.

The local quality gate at the time of this report was:

- 107 tests passed;
- Ruff passed;
- mypy passed on `src`;
- `git diff -check` passed.

Table 6: Final project deliverables.

Deliverable	Location or description
Source code	PatchPilot package under <code>src/patchpilot/</code> .
Controlled benchmark	<code>benchmarks/</code> .
Mutmut benchmark	<code>generated_benchmarks/mutmut_algorithms/</code> .
Demo frontend	<code>demo/streamlit_app.py</code> and hosted Hugging Face Space.
Evaluation scripts	Evaluation, benchmark-generation, and smoke-test scripts under <code>scripts/</code> .
Result summaries	Committed CSV files under <code>results/</code> .
Report assets	Generated figures and tables under <code>docs/report_assets/</code> .
Final report	<code>docs/PatchPilot_Final_Report.pdf</code> after review and compilation.

Limitations

PatchPilot has several important limitations.

- It targets small Python repositories with pytest suites, not large multi-service systems.
- The main mutmut benchmark contains 20 tasks, which limits statistical power.
- The controlled benchmark is saturated and should not be used as evidence of superiority.
- The live model backend uses a small local model, so failures may reflect model capacity as well as agent design.
- QuixBugs is used only as a smoke test.
- SWE-bench Lite was not completed because of local WSL/Docker instability.
- The current policy repairs source code only; it does not implement deeper project-level reasoning such as dependency upgrades, multi-file refactors, or issue-discussion interpretation.

These limitations are acceptable for a Master’s project prototype, but they define the boundary of the claims. The project demonstrates a working bounded repair agent and a reproducible evaluation pipeline. It does not claim production-grade repair performance.

Future Work

The most direct improvement is to expand the mutmut benchmark and repeat the comparison across more tasks. A larger benchmark would make paired statistical tests more meaningful. A second improvement is stronger fault localization: the policy could summarize failing assertions, rank candidate functions, and include more targeted context before patch synthesis. A third direction is model comparison. Because the backend is model-swappable, the same tasks could be rerun with stronger local or hosted code models.

Longer-term work could add multi-file repair, dependency-aware environments, richer diff validation, and a more robust path to official SWE-bench evaluation. The current architecture already separates policy, execution, and benchmark management, so these additions can be made without removing the safety boundary.

Conclusion

PatchPilot delivers a complete bounded agentic repair workflow for Python tasks. It prepares isolated workspaces, reproduces failures, inspects source code through restricted tools, applies controlled patches, verifies with tests, records traces, and exposes the workflow through a live demo. The final repository contains the implementation, benchmarks, generated mutmut tasks, evaluation scripts, result summaries, and report assets.

The empirical result is modest but meaningful. On the primary mutmut benchmark, the full agent achieves the highest repair rate among the compared conditions: 40% versus 30% for one-shot and 25% for no-retry. The result is not statistically significant at the current benchmark size, so it should be read as evidence that the full loop helped in this setting, not as proof of general superiority. The main contribution is therefore a safe, reproducible, and inspectable agentic repair system that can be extended in future work.

References

- [1] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama. QuixBugs: A Multi-Lingual Program Repair Benchmark Set Based on the Quixey Challenge. *SPLASH Companion*, 2017. <https://jkoppel.github.io/QuixBugs/>
- [2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*, 2024. <https://arxiv.org/abs/2310.06770>
- [3] SWE-bench Team. SWE-bench Lite documentation. <https://www.swebench.com/lite.html>
- [4] mutmut contributors. mutmut - Python mutation tester documentation. <https://mutmut.readthedocs.io/>
- [5] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, and others. Qwen2.5-Coder Technical Report. arXiv:2409.12186, 2024. <https://arxiv.org/abs/2409.12186>